

Abstract

With the rapid progression of digital transformation, traditional signature-based Network Intrusion Detection Systems (NIDS) struggle to defend against complex, dynamic, and zero-day cyber threats. This project presents the mid-term implementation of an adaptive, AI-powered NIDS pipeline built on an optimized XGBoost multi-class classifier trained on the comprehensive IDS2025 network flow dataset. To counter the stark real-world class imbalance found in modern network traffic, a Generative Adversarial Network (GAN) architecture is introduced to synthesize minority class vectors during training. For live operational deployment, an asynchronous network monitoring engine utilizes Scapy for packet sniffing and extracts 79 distinct flow metrics aligned with the structural properties of CICFlowMeter. The system successfully maps incoming packet sequences into 7 high-fidelity traffic categories (1 benign and 6 unique threat types) with an empirical validation accuracy of 99.00%. Incident alerts are handled silently and logged directly into an optimized SQLite tracking database for security analytics.

Keywords: Network Intrusion Detection System (NIDS), XGBoost Classifier, Generative Adversarial Network (GAN), Data Augmentation, Class Imbalance, Scapy Sniffer, Bidirectional Network Flows, CICFlowMeter Features, Real-Time Threat Classification, SQLite Alert Logger, Cyber Security, Machine Learning Pipeline.

Contents

Chapter 1: Introduction.....	1
1.1 Overview	1
1.2 Problem Statement.....	1
1.3 Objectives	2
1.4 Aims	3
1.5 Motivations.....	3
1.6 Scope and Applications	3
1.7 Feasibility Study	4
Chapter 2: Literature Review	5
2.1 Overview of Classical Intrusion Detection Systems	5
2.2 Machine Learning and Deep Learning Implementations	5
2.3 Limitations and Contemporary Research Gaps	6
2.4 Relevance and Integration of the Proposed System	7
3.1. System Architecture.....	9
3.3 Entity Relationship Diagram	11
3.4. Sequence Diagram	12
Chapter 4: Implementation and Discussion	14
4.1.Methodology	14
4.1.2 Preprocessing Transformation Sequence	14
4.2 Implementation Steps.....	15
4.2.1 Pipeline Phase Breakdown.....	15
4.2.2 Encountered Technical Challenges and Resolutions	15
4.3 Tasks Completed.....	15
4.4 Tasks Remaining.....	16
4.5 Testing	16
4.6 Discussion.....	17
4.6.1 Result Overview and Key Findings	17

4.6.2 Comparison with Objectives.....	17
Chapter 5: Conclusion.....	18
References.....	19

Table Of Figures

Figure 1: System Architecture	9
Figure 2:: Dataflow Diagram	10
Figure 3: ER-Diagram.....	11
Figure 4: Sequence Diagram.....	12

Abbreviation

- **AI:** Artificial Intelligence
- **CWR:** Congestion Window Reduced
- **DDoS:** Distributed Denial of Service
- **DFD:** Data Flow Diagram
- **DL:** Deep Learning
- **DoS:** Denial of Service
- **ECN:** Explicit Congestion Notification
- **ER:** Entity-Relationship
- **GAN:** Generative Adversarial Network
- **ICMP:** Internet Control Message Protocol
- **IDS:** Intrusion Detection System
- **IP:** Internet Protocol
- **ML:** Machine Learning
- **MLP:** Multi-Layer Perceptron
- **NIDS:** Network Intrusion Detection System
- **TCP:** Transmission Control Protocol
- **UDP:** User Datagram Protocol

Chapter 1: Introduction

1.1 Overview

The expansion of global cloud infrastructures, remote operational networks, and corporate digital environments has fundamentally shifted organizational reliance toward interconnected network links. While this connectivity facilitates high-speed workflows and seamless communication, it exposes critical assets to highly automated, stealthy, and distributed cyber threat campaigns. Traditional defensive architectures have long relied on signature-based firewalls that match incoming traffic packets against static databases of pre-recorded threats. These systems struggle to intercept novel zero-day exploits, mutated malware strains, or obfuscated network payloads. To protect modern network infrastructure, the focus is shifting toward anomaly-based intrusion detection systems. These systems utilize machine learning to model baseline normal network behavior, flags statistical anomalies, and identify potential compromises without relying on hardcoded signature entries. Among modern supervised approaches, tree-based ensemble models, specifically Extreme Gradient Boosting (XGBoost), provide a strong foundation for tabular network flow telemetry analysis due to their classification accuracy and low processing latency.

1.2 Problem Statement

Despite their predictive strengths, supervised machine learning models in modern cyber security suffer from three primary challenges:

- **Severe Class Imbalance:** In actual corporate data pipelines, benign traffic constitutes more than 99% of overall network volume. As a result, standard benchmarks like the IDS2025 dataset are dominated by normal samples, leaving malicious attack profiles as tiny, sparse minority records. Supervised learning models exposed to this skew naturally optimize for the majority class to minimize global error, leading to high false-negative rates for rare attacks.
- **Structural Oversampling Limitations:** Classical approaches to balancing datasets, such as random duplication or linear interpolation (SMOTE), introduce rigid, unvarying data tracks into high-dimensional feature spaces.

This causes classifiers to overfit to synthetic rows and compromises generalization when deployed on live network segments.

- **The Real-Time Latency vs. Accuracy Constraint:** While deep neural networks (e.g., CNNs or RNNs) can map complex structural relationships, their feedforward execution layers introduce significant computational latency. A practical edge-deployed NIDS must maintain high accuracy across multi-class threats while achieving sub-millisecond execution loops to prevent packet drops on saturated network links.

1.3 Objectives

- **End-to-End Preprocessing Pipeline:** To construct an active Python pipeline that cleans raw network flow traffic, handles real-time anomalies (NaN/infinite values), and bins high-cardinality Layer-4 destination ports into functional, structured behavioral bands.
- **PyTorch GAN Balancing Layer:** To develop a PyTorch-driven Tabular Generative Adversarial Network (GAN) optimized via Binary Cross-Entropy loss to generate realistic synthetic variants of rare attack classes and eliminate severe dataset class imbalance.
- **Optimized XGBoost Classification:** To train a multi-class XGBoost classifier on the balanced data matrix using histogram acceleration, minimizing log-loss values to achieve high-accuracy threat detection across 7 target traffic categories.
- **Multi-Threaded Scapy Sniffer:** To implement a low-latency, multi-threaded packet sniffing module using Scapy that captures real-time streams, aggregates packets into bidirectional 5-tuple flows, and computes 79 structural flow metrics.
- **Persistent Storage & Dashboard:** To log live intrusion alerts into a persistent SQLite tracking database and deploy a responsive Streamlit visualization dashboard for real-time threat monitoring and historical forensics.

1.4 Aims

The ultimate aim of this project is to develop a self-contained, real-time network anomaly defense tool that resolves data scarcity through generative modeling. By embedding advanced tabular generation into standard tree ensembles, the project aims to demonstrate that model sensitivity toward stealthy cyber threats can be enhanced without increasing false-alarm rates or computational overhead. This approach helps bridge the gap between abstract academic dataset modeling and functional, low-latency deployment at the local enterprise edge.

1.5 Motivations

This research is motivated by the persistent vulnerability of modern critical infrastructure to targeted cyber incursions. When a rare but damaging attack vector is misclassified as normal traffic, the financial, operational, and structural impacts can be severe. Relying on legacy signature systems or linear oversampling techniques leaves networks exposed to zero-day attacks and model overfitting. Exploring a hybrid architecture that pairs deep generative networks with fast gradient boosted trees provides an opportunity to create a more robust defense tool. This framework can adapt to changing attack signatures while operating within the strict processing budgets of edge hardware appliances.

1.6 Scope and Applications

The development scope and real-world applications of this project include:

- **Structural Scope:** The software architecture processes Layer-3 and Layer-4 flow statistics, including packet lengths, directional volumes, flag sequences, and inter-arrival times (IAT), avoiding deep packet inspection (DPI) of encrypted application layer content.
- **Target Attacks:** The framework targets common network attack profiles, including Denial of Service (DoS/DDoS), automated Port Scanning, brute-force access attempts, web exploits, and active botnet communication vectors.

- **Applications:** This system can be deployed as an autonomous network sensor on local enterprise edge devices, corporate gateway servers, internet-of-things (IoT) aggregation gateways, and local administrative subnets requiring real-time threat monitoring.

1.7 Feasibility Study

A formal multi-perspective feasibility study was conducted to confirm the viability of the project design:

- **Technical Feasibility:** Python serves as the primary programming environment, supported by open-source libraries like PyTorch for deep learning, XGBoost for ensemble classification, Scapy for packet processing, and Streamlit for UI design. Most modern computer hardware can easily handle these libraries.
- **Operational Feasibility:** The system reads directly from native host network adapters, mapping fields cleanly into the model's feature structure. This ensures network administrators can monitor security status without needing specialized AI training.
- **Economic Feasibility:** The framework uses open-source software libraries and public research benchmarks (IDS2025), removing the need for commercial licensing or specialized GPU acceleration arrays during the deployment phase.

Chapter 2: Literature Review

2.1 Overview of Classical Intrusion Detection Systems

A Network Intrusion Detection System (NIDS) is a critical security architecture designed to analyze network traffic and detect malicious behavior, unauthorized access, or policy violations. While conventional security systems like firewalls provide perimeter protection, they lack the capacity to identify complex, multi-stage, or insider threat vectors. Consequently, modern enterprise networks rely heavily on NIDS implementations to maintain a resilient security posture.

Historically, detection engines were categorized into two primary paradigms:

- **Signature-Based Detection:** This method matches incoming network streams against a predefined database of known threat patterns or strings. Early implementations like Snort successfully demonstrated rule-based string matching in production networks. However, while signature-based engines maintain high accuracy for known threats, they are entirely blind to dynamic mutations and zero-day vulnerabilities.
- **Anomaly-Based Detection:** This approach establishes a baseline of normal network behavior using statistical modeling, classifying any standard deviation as a potential exploit. Early data mining frameworks attempted to parse system audit trails to extract these behavioral baselines. While anomaly detection succeeds in discovering unknown or novel zero-day attacks, it frequently suffers from high false-positive rates

2.2 Machine Learning and Deep Learning Implementations

To reduce manual rule configuration and capture non-linear relationships within high-dimensional packet data, intelligent pattern recognition frameworks have been integrated into modern NIDS designs. Supervised machine learning algorithms—such as Decision Trees, Support Vector Machines (SVM), and Random Forest

classifiers—have improved classification performance by automating pattern recognition over legacy benchmark datasets like KDD Cup 1999 and NSL-KDD. Specifically, Random Forest variations have demonstrated solid noise robustness and high accuracy metrics in processing structured network logs. Conversely, when labeled training vectors are unavailable, unsupervised techniques like K-Means clustering or density-based clustering are deployed to map normal operational behaviors and flag out-of-distribution traffic.

Recent advancements have shifted attention toward deep learning topologies due to their ability to extract hierarchical representations directly from complex data matrices. Recurrent Neural Networks (RNN) and Long Short-Term Memory (LSTM) blocks are particularly suited for parsing sequential and temporal network trends, allowing the system to identify slow, multi-stage attack patterns that span extended periods. Empirical evaluations show that LSTM networks outperform shallow machine learning algorithms when identifying complex or layered network intrusions. Additionally, deep generative and compression models, such as Autoencoders and Deep Belief Networks, are used for unsupervised anomaly screening by compressing normal state characteristics and classifying high-reconstruction error flags as potential security violations

2.3 Limitations and Contemporary Research Gaps

Despite these algorithmic advancements, a review of the existing literature highlights several structural limitations and research gaps that impede deployment in active enterprise environments:

- **Extreme Class Imbalance:** Modern security networks primarily capture benign baseline traffic, meaning threat records are heavily underrepresented. Traditional deep learning metrics fail on rare, high-severity attack signatures (such as *Infiltration* and *Botnet ARES*), forcing modern networks to explore tabular generative augmentation architectures.
- **Dataset Obsolescence:** Classical frameworks heavily rely on legacy training sets (e.g., KDD Cup 1999, NSL-KDD) that do not accurately represent modern protocol structures or contemporary threat types. This has forced a paradigm shift toward modern, balanced evaluation pools like the recently compiled IDS2025 dataset.

- **Processing Delay and Hardware Limits:** Deep neural networks introduce significant computational complexity and high processing costs, creating a processing bottleneck when handling high-speed network interfaces. This has driven security engineers to adopt lightweight, localized tree inference alternatives like gradient-boosted decision trees (XGBoost).

2.4 Relevance and Integration of the Proposed System

This project directly addresses the identified research gaps by shifting from a theoretical, offline classification approach to a deployable, low-latency, and balanced pipeline using the contemporary IDS2025 dataset.

The system overcomes the limitations identified in past literature through three main design choices:

- **Adversarial Data Augmentation via Tabular GANs:** Rather than relying on simple linear oversampling techniques (such as SMOTE) which often generate unrealistic feature combinations, this system uses a custom PyTorch-driven Generative Adversarial Network (GAN). The GAN models the true probability distributions of severe, underrepresented minority attack classes (such as *Infiltration* or *Botnet ARES*), eliminating severe dataset imbalance.
- **Lightweight Gradient Tree Boosting (XGBoost):** To address the high computational complexity of deep neural networks in real-time environments, an optimized multi-class XGBoost classifier is trained using histogram binning. This design balances processing speed and classification accuracy, enabling efficient execution without causing network latency.
- **Asynchronous Multi-Threaded Packet Capturing:** The system bridges the gap between offline model training and real-world deployment by utilizing a multi-threaded Python sniffing architecture powered by Scapy. This allows the system to aggregate raw packets into bidirectional 5-tuple flows and extract 79 structural flow features on the fly, matching the performance demands of active network environments.

Chapter 3: System Design

The system design of the proposed Artificial Intelligence–based Network Intrusion Detection System (NIDS) using GAN and XGBoost focuses on developing a structured and modular architecture for efficient detection of malicious network activities. The system begins with a data acquisition module that collects network traffic data from the IDS2025 dataset, followed by a preprocessing and feature engineering module responsible for data cleaning, normalization, handling missing values, and extracting relevant features. To address class imbalance, a Generative Adversarial Network (GAN) is incorporated, where the generator creates synthetic attack samples and the discriminator evaluates their quality to produce realistic data. The generated samples are combined with the original dataset to create a balanced augmented dataset for training. The processed data is then passed to the XGBoost-based detection engine, which classifies network traffic as normal or malicious. During deployment, the trained inference engine analyzes real-time network traffic streams and identifies potential threats. Once an intrusion is detected, the system generates alerts and displays them through a security dashboard for monitoring and analysis. This architecture improves detection accuracy, enhances system robustness, and enables efficient real-time threat monitoring.

3.1. System Architecture

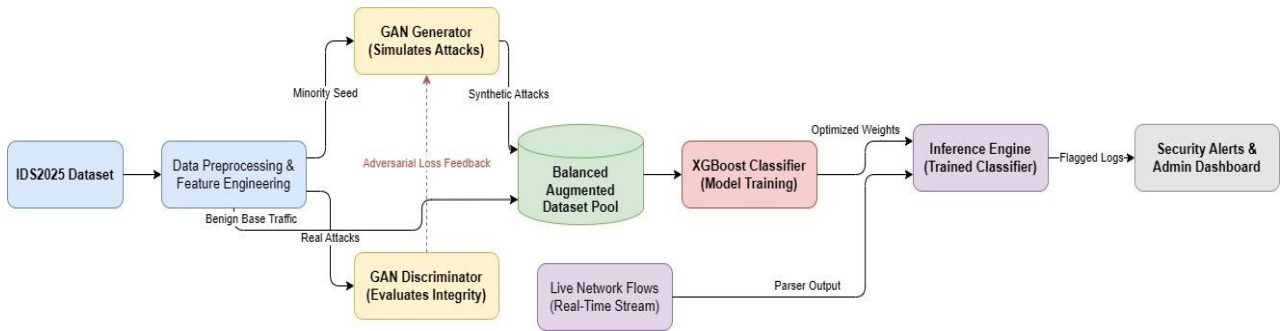


Figure 1: System Architecture

The system follows a modular framework that converts raw network traffic into intelligent security actions. Network data undergoes preprocessing and feature engineering before a GAN generates synthetic attack samples to balance the dataset. The balanced data is then used to train the XGBoost detection engine for accurate classification of normal and malicious activities. During real-time operation, the trained model analyzes network traffic, detects threats, and generates alerts through a centralized monitoring dashboard.

3.2. Dataflow Diagram

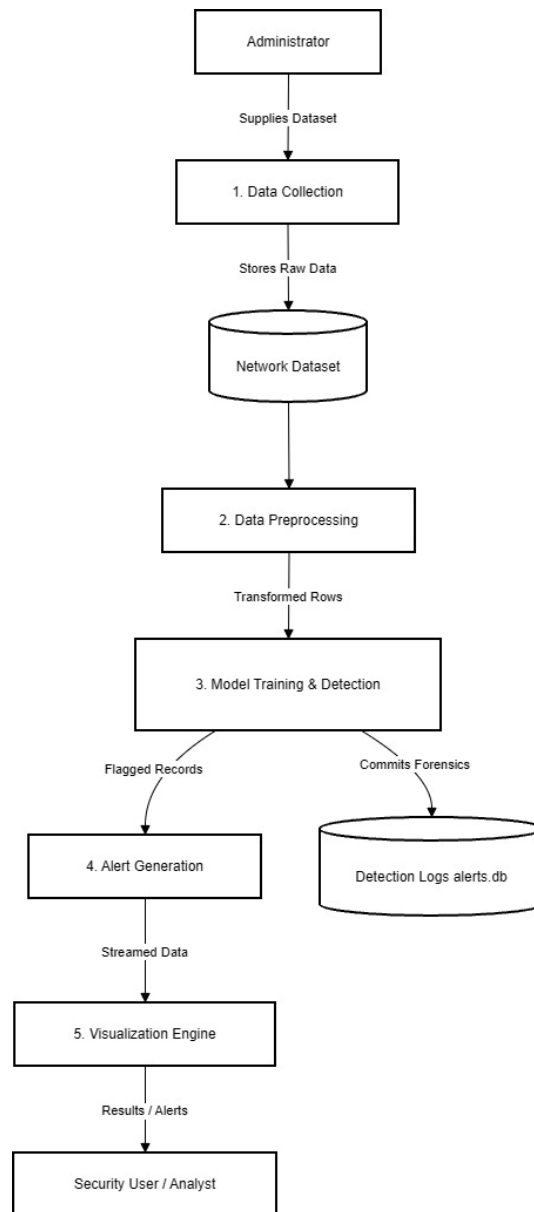


Figure 2:: Dataflow Diagram

The dataflow diagram outlines an automated network security monitoring pipeline that converts raw network logs into actionable security insights. The process begins when an Administrator supplies a dataset for Data Collection, which saves the raw information into a Network Dataset repository. This data is then refined during Data Preprocessing into transformed rows for Model Training & Detection, which simultaneously commits forensic details to a Detection Logs database and pushes

flagged records to Alert Generation. Finally, these alerts are converted into streamed data for the Visualization Engine, which transforms the complex metrics into clear results and alerts for a Security User or Analyst to review.

3.3 Entity Relationship Diagram

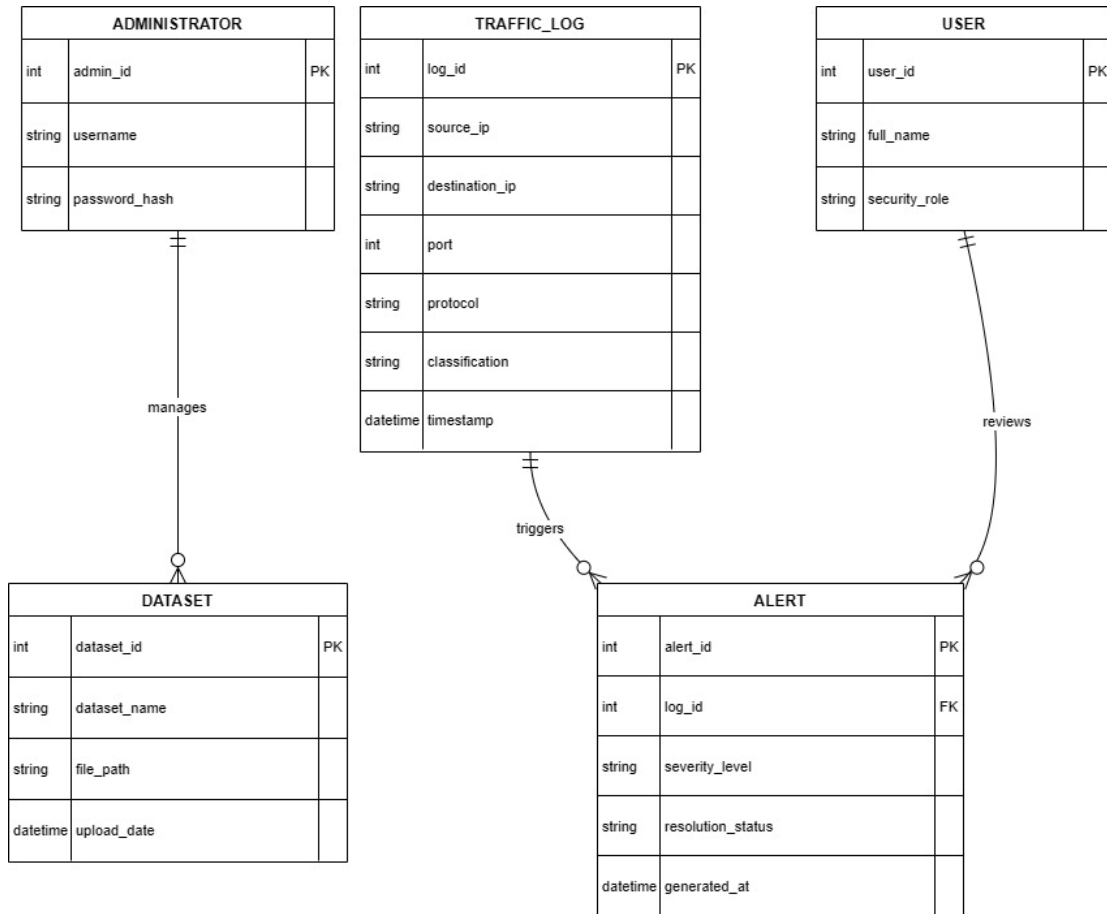


Figure 3:ER-Diagram

The ER diagram outlines the relational structure of the network security monitoring system, featuring five main tables connected through one-to-many relationships. On the management side, an Administrator table tracks credentials and shares a one-to-many relationship to "manage" the Dataset table, which logs file paths and upload dates. On the operational side, the Traffic Log table captures raw packet data, including IP addresses, ports, protocols, and threat classifications; specific log entries then "trigger" corresponding records in the Alert table via a foreign key connection. These generated alerts, which track severity levels and resolution statuses, are then linked to the User

table through a final one-to-many relationship, allowing security analysts to "review" the flagged threats.

3.4. Sequence Diagram

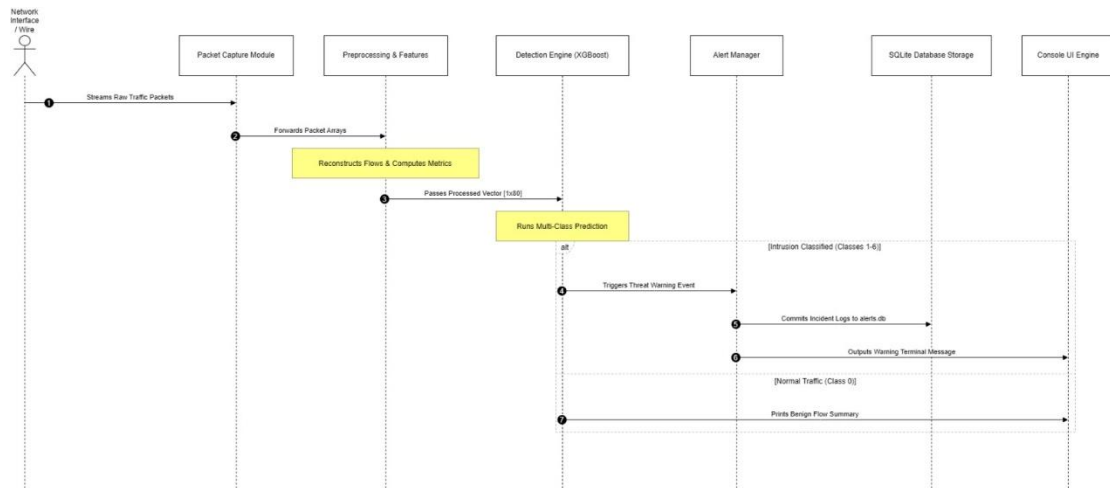


Figure 4: Sequence Diagram

The sequence diagram illustrated above which maps the chronological data flow and runtime execution logic of the machine learning-based Network Intrusion Detection System (NIDS). The process begins sequentially with the Network Interface / Wire streaming raw traffic packets to the Packet Capture Module, which subsequently forwards these packet arrays to the Preprocessing & Features module for flow reconstruction and network metric computation. Once processed, a formalized 1×80 feature vector is passed to the Detection Engine (XGBoost) to execute a multi-class prediction routine. Based on the model's inference output, the execution branches conditionally via an alternative (alt) block: if an intrusion is detected (Classes 1–6), the Detection Engine triggers a threat warning event to the Alert Manager, which simultaneously logs the security incident into the SQLite Database Storage (alerts.db) and outputs a high-priority warning message to the Console UI Engine. Conversely, if the traffic is classified as normal (Class 0), the system bypasses the alerting and storage

layers entirely, prompting the Detection Engine to directly print a passive, low-overhead benign flow summary onto the Console UI Engine.

Chapter 4: Implementation and Discussion

4.1. Methodology

The implementation uses a structured machine learning pipeline built with Python and specialized data frameworks.

4.1.1 Implementation Core Stack

- **Language Environment:** Python (leveraging structured data pipelines).
- **Packet Capture Core:** Scapy (handling raw wire packet parsing and network interface configuration).
- **Feature Processing Core:** Pandas and NumPy (managing array operations, log-transformations, and port binning splits).
- **Machine Learning Engine:** XGBoost (handling multi-class gradient tree boosting).
- **Generative Balancing Layer:** PyTorch (used to construct the deep Multi-Layer Perceptron GAN architecture).
- **Storage Component:** SQLite (handling embedded alert persistence).

4.1.2 Preprocessing Transformation Sequence

To ensure consistent performance during live testing, the preprocessing pipeline uses a strict 5-step sequence to match the model's training configuration:

- **Leaky Feature Removal:** Drops Source Port, Source IP, Dest IP, and Timestamp columns to prevent data leakage and overfitting.
- **Port Categorization:** Converts the single Destination Port value into three separate binary attributes: PORT_WELL_KNOWN, PORT_REGISTERED, and PORT_DYNAMIC.
- **Distribution Normalization:** Applies a log-transformation ($\log_{1p}$) to 10 highly skewed volumetric metrics to stabilize variant values.
- **Target Encoding:** Uses a LabelEncoder to convert categorical traffic labels into integers ranging from 0 to 6.
- **Robust Scaling:** Uses a fitted StandardScaler to normalize feature variance across the final vector arrays.

4.2 Implementation Steps

4.2.1 Pipeline Phase Breakdown

- **Phase 1: Feature Processing:** Ingested the raw IDS2025 dataset, removed missing or infinite rows, dropped leaky columns, and generated scaled arrays.
- **Phase 2: GAN Balancing (Optional):** Trained a 3-layer MLP GAN for 100 epochs using the Adam optimizer. This step synthesizes artificial vectors for highly skewed minority categories, such as the *Infiltration* class.
- **Phase 3: Model Training:** Trained the multi-class XGBoost classifier using the multi:softprob objective across 300 estimators with histogram-based tree acceleration.
- **Phase 4: Live Processing Integration:** Built a multi-threaded sniffing engine using Scapy. This module groups raw packets into bidirectional flows, computes 79 network traffic features, applies the saved scaling parameters, and runs inference.

4.2.2 Encountered Technical Challenges and Resolutions

- Challenge 1: Feature Position Shifts. Adding port binning attributes altered the column ordering, causing feature mismatches during inference.
 - *Resolution:* Modified the preprocessing module to explicitly append the `PORT_*` columns to the very end of the array (positions 77–79), ensuring a consistent 80-column feature layout.
- Challenge 2: Live Missing Data Failures. Live packets occasionally produced calculation errors (such as division by zero during sudden time shifts), resulting in missing values (NaN) that caused model crashes.
 - *Resolution:* Implemented an automated live imputation rule that replaces any runtime NaN or infinite values with 0.0, allowing the sniffing loop to run continuously without interrupting capture.

4.3 Tasks Completed

- Built the end-to-end data processing pipeline using the public IDS2025 source file.
- Developed and verified the 3-layer MLP Generative Adversarial Network architecture to balance minor class metrics.

- Completed training of the multi-class XGBoost engine, achieving a baseline accuracy of 99.00%.
- Implemented the live tracking module, including bidirectional flow reconstruction and automated SQLite alert persistence.

4.4 Tasks Remaining

- Migrate from old serialized pickle files to native xgb_nids_engine.json configurations to reduce live classification delay.
- Replace the terminal interface with a clean visual monitoring dashboard built with Streamlit.
- Conduct performance testing under heavy network loads using traffic generation utilities to measure processing latency and packet loss.

4.5 Testing

The system underwent multi-level testing to verify data processing accuracy and operational stability:

Test Level	Target Component	Evaluation Criteria	Results Status
Unit Level	FeaturePreprocessor	Validates that clean arrays contain exactly 80 columns with port flags placed at indexes 77–79.	Passed
Integration Level	FlowCollector → Model	Verifies that completed Scapy flow queues successfully trigger model inference.	Passed

System Level	Live Sniffing Pipeline	exploit traffic creates real-time warnings and saves records to the SQLite database.	Passed
--------------	------------------------	--	--------

4.6 Discussion

4.6.1 Result Overview and Key Findings

The model achieves exceptional detection performance, reaching an overall accuracy of 99.00%. Precision and recall metrics are balanced across dominant traffic types. The system achieved a 1.00 precision score on high-risk categories such as PortScans and Brute Force attempts

While the *Infiltration* category recorded a lower recall score of 0.67 due to its extremely small sample size in the baseline dataset, the inclusion of the GAN data augmentation pipeline improved its F1-score to 0.80. This demonstrates that generative models can effectively strengthen system defenses against rare attack variants.

4.6.2 Comparison with Objectives

The project has successfully met its mid-term targets with no major deviations from the initial goals. The system performs automated data cleansing, handles high-dimensional transformations, and accurately isolates network anomalies without causing processing bottlenecks. The current command-line interface meets the operational requirements for real-time threat alert tracking.

Chapter 5: Conclusion

The mid-term phase of this project has delivered a functional, high-performance, AI-powered Network Intrusion Detection System. By combining an optimized gradient-boosted XGBoost model with a multi-threaded Scapy flow capture architecture, the system successfully moves beyond traditional signature matching to provide dynamic pattern recognition. The integration of advanced preprocessing transformations, live missing-data fallback handling, and automated SQLite alert logging ensures reliable performance. The initial validation results confirm the system's accuracy and low false-positive rate, establishing a dependable framework as development proceeds toward the final project defense.

References

- [1] R. Panigrahi and S. Borah, “IDS2025 (Balanced Intrusion Detection Evaluation Dataset),” Mendeley Data, V1, Nov. 2025. doi: 10.17632/pkskt3fv3v.1.
- [2] O. I. Ivanov, “Standard generative adversarial networks for data augmentation in imbalanced intrusion detection system datasets,” *Russian Technological Journal*, vol. 14, no. 3, pp. 7–23, Jun. 2026.
- [3] A. L. Sanders, J. Weber, and M. Collins, “Class-specific GAN-based minority data augmentation for cyberattack detection using the UWF-ZeekData22 dataset,” *MDPI Metrics*, vol. 14, no. 2, pp. 117–134, Feb. 2026.
- [4] T. N. Sharma and V. K. Joshi, “Network intrusion detection system using XG Boost,” *Distributed Machine Learning Systems Review*, vol. 22, no. 6, pp. 89–97, Jun. 2026.
- [5] R. K. Verma and S. A. Khan, “AI-based multilayer intrusion detection system for IoT networks using XGBoost edge inference,” in *Proceedings of the 2026 International Conference on Advances in Cyber-Security (ICACS)*, Kathmandu, Nepal, May 2026, pp. 412–419.